

String Similarity II

Paweł Gawrychowski

University of Wrocław

While $\mathcal{O}(n + D^2)$ seems nice from a purely practical point of view, it can still be $\Theta(n^2)$ in the worst case. So, can we beat this complexity, at least in theory?

Masek and Paterson 1980

Edit distance between two strings over an alphabet of constant size can be computed in time $\mathcal{O}(\frac{n^2}{\log n})$.

Masek and Paterson 1980

Edit distance between two strings over any alphabet can be computed in time $\mathcal{O}(\frac{n^2 \log \log n}{\log n})$.

We will describe this technique for the LCS problem, as this makes things a bit simpler.

While $\mathcal{O}(n + D^2)$ seems nice from a purely practical point of view, it can still be $\Theta(n^2)$ in the worst case. So, can we beat this complexity, at least in theory?

Masek and Paterson 1980

Edit distance between two strings over an alphabet of constant size can be computed in time $\mathcal{O}(\frac{n^2}{\log n})$.

Masek and Paterson 1980

Edit distance between two strings over any alphabet can be computed in time $\mathcal{O}(\frac{n^2 \log \log n}{\log n})$.

We will describe this technique for the LCS problem, as this makes things a bit simpler.

While $\mathcal{O}(n + D^2)$ seems nice from a purely practical point of view, it can still be $\Theta(n^2)$ in the worst case. So, can we beat this complexity, at least in theory?

Masek and Paterson 1980

Edit distance between two strings over an alphabet of constant size can be computed in time $\mathcal{O}(\frac{n^2}{\log n})$.

Masek and Paterson 1980

Edit distance between two strings over any alphabet can be computed in time $\mathcal{O}(\frac{n^2 \log \log n}{\log n})$.

We will describe this technique for the LCS problem, as this makes things a bit simpler.

While $\mathcal{O}(n + D^2)$ seems nice from a purely practical point of view, it can still be $\Theta(n^2)$ in the worst case. So, can we beat this complexity, at least in theory?

Masek and Paterson 1980

Edit distance between two strings over an alphabet of constant size can be computed in time $\mathcal{O}(\frac{n^2}{\log n})$.

Masek and Paterson 1980

Edit distance between two strings over any alphabet can be computed in time $\mathcal{O}(\frac{n^2 \log \log n}{\log n})$.

We will describe this technique for the LCS problem, as this makes things a bit simpler.

We will introduce an interesting (and, actually, not-that-theoretical) technique due to Arlazarov, Dinic, Kronrod, and Faradzev (1970).

Claim

If the dynamic programming matrix is really large, it seems reasonable that some of its small fragments “look” the same. Hence maybe we could precompute some data, and then speed-up the computation?

...but what does it mean that the fragments “look” the same?

We will introduce an interesting (and, actually, not-that-theoretical) technique due to Arlazarov, Dinic, Kronrod, and Faradzev (1970).

Claim

If the dynamic programming matrix is really large, it seems reasonable that some of its small fragments “look” the same. Hence maybe we could precompute some data, and then speed-up the computation?

...but what does it mean that the fragments “look” the same?

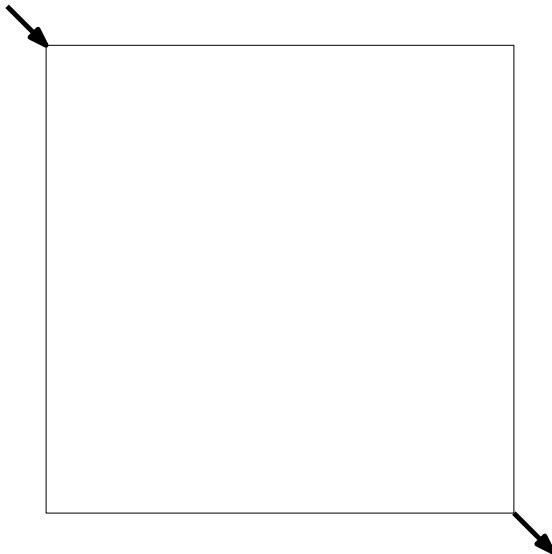
We will introduce an interesting (and, actually, not-that-theoretical) technique due to Arlazarov, Dinic, Kronrod, and Faradzev (1970).

Claim

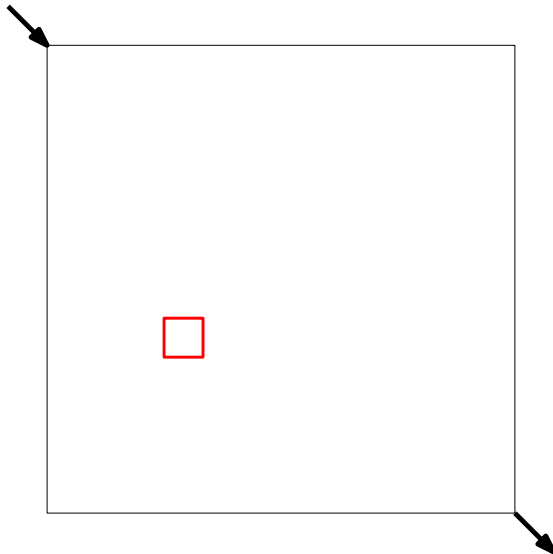
If the dynamic programming matrix is really large, it seems reasonable that some of its small fragments “look” the same. Hence maybe we could precompute some data, and then speed-up the computation?

...but what does it mean that the fragments “look” the same?

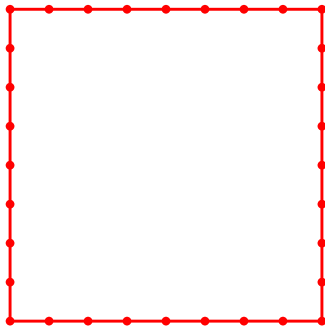
Computing LCS



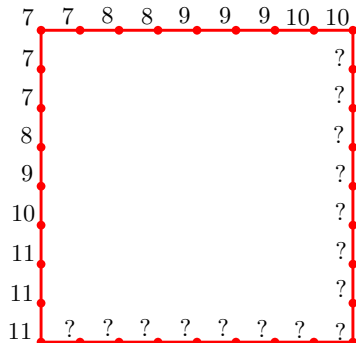
Computing LCS



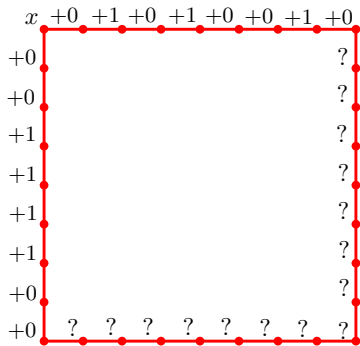
Computing LCS



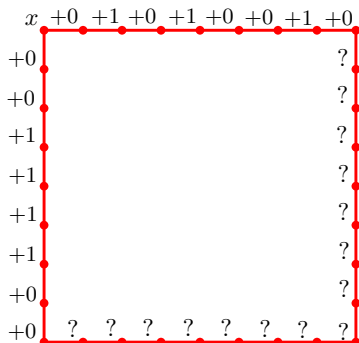
Computing LCS



Computing LCS



Computing LCS



The values on the lower and right boundary depend only on the sequence of $+0/1$ and the corresponding fragments of the strings.

Instead of computing all values in the middle, we immediately evaluate the values on the lower and right boundary using:

- the value of x ,
- the sequences of $+0/1$ on the left and top boundary,
- the corresponding fragments of s and t .

The value of x is not important: we can think that $x = 0$, perform the computation, and then add the original value of x to all results.

Instead of computing all values in the middle, we immediately evaluate the values on the lower and right boundary using:

- the value of x ,
- the sequences of $+0/1$ on the left and top boundary,
- the corresponding fragments of s and t .

The value of x is not important: we can think that $x = 0$, perform the computation, and then add the original value of x to all results.

Instead of computing all values in the middle, we immediately evaluate the values on the lower and right boundary using:

- the value of x ,
- the sequences of $+0/1$ on the left and top boundary,
- the corresponding fragments of s and t .

The value of x is not important: we can think that $x = 0$, perform the computation, and then add the original value of x to all results.

Instead of computing all values in the middle, we immediately evaluate the values on the lower and right boundary using:

- the value of x ,
- the sequences of $+0/1$ on the left and top boundary,
- the corresponding fragments of s and t .

The value of x is not important: we can think that $x = 0$, perform the computation, and then add the original value of x to all results.

Instead of computing all values in the middle, we immediately evaluate the values on the lower and right boundary using:

- the value of x ,
- the sequences of $+0/1$ on the left and top boundary,
- the corresponding fragments of s and t .

The value of x is not important: we can think that $x = 0$, perform the computation, and then add the original value of x to all results.

Say that the block is $B \times B$. How many different situations can we encounter, assuming that $s, t \in \{a, b\}^*$?

Lemma

The number of different situations is 2^{4B} .

We precompute the results for each possible situation. Then we move block-by-block, and use the precomputed data to quickly determine the next lower and right boundary. What is the resulting time complexity?

$$2^{4B} B^2 + \frac{n^2}{B}$$

Setting $B = \frac{1}{4} \log n$ gives us $\mathcal{O}(\frac{n^2}{\log n})$.

Say that the block is $B \times B$. How many different situations can we encounter, assuming that $s, t \in \{a, b\}^*$?

Lemma

The number of different situations is 2^{4B} .

We precompute the results for each possible situation. Then we move block-by-block, and use the precomputed data to quickly determine the next lower and right boundary. What is the resulting time complexity?

$$2^{4B} B^2 + \frac{n^2}{B}$$

Setting $B = \frac{1}{4} \log n$ gives us $\mathcal{O}(\frac{n^2}{\log n})$.

Say that the block is $B \times B$. How many different situations can we encounter, assuming that $s, t \in \{a, b\}^*$?

Lemma

The number of different situations is 2^{4B} .

We precompute the results for each possible situation. Then we move block-by-block, and use the precomputed data to quickly determine the next lower and right boundary. What is the resulting time complexity?

$$2^{4B} B^2 + \frac{n^2}{B}$$

Setting $B = \frac{1}{4} \log n$ gives us $\mathcal{O}(\frac{n^2}{\log n})$.

Say that the block is $B \times B$. How many different situations can we encounter, assuming that $s, t \in \{a, b\}^*$?

Lemma

The number of different situations is 2^{4B} .

We precompute the results for each possible situation. Then we move block-by-block, and use the precomputed data to quickly determine the next lower and right boundary. What is the resulting time complexity?

$$2^{4B} B^2 + \frac{n^2}{B}$$

Setting $B = \frac{1}{4} \log n$ gives us $\mathcal{O}(\frac{n^2}{\log n})$.

Say that the block is $B \times B$. How many different situations can we encounter, assuming that $s, t \in \{a, b\}^*$?

Lemma

The number of different situations is 2^{4B} .

We precompute the results for each possible situation. Then we move block-by-block, and use the precomputed data to quickly determine the next lower and right boundary. What is the resulting time complexity?

$$2^{4B} B^2 + \frac{n^2}{B}$$

Setting $B = \frac{1}{4} \log n$ gives us $\mathcal{O}(\frac{n^2}{\log n})$.

Bit packing

If you believe that one can pack $\log n$ bits in a single machine word, and operate on such words in constant time (i.e., multiply, add, xor, extract a given bit,...), we can actually achieve $\mathcal{O}(\frac{n^2}{\log^2 n})$ (for the constant alphabet) by storing the $\log n$ values on each boundary in a single word.

This was not done in the original paper!

Bit packing

If you believe that one can pack $\log n$ bits in a single machine word, and operate on such words in constant time (i.e., multiply, add, xor, extract a given bit,...), we can actually achieve $\mathcal{O}(\frac{n^2}{\log^2 n})$ (for the constant alphabet) by storing the $\log n$ values on each boundary in a single word.

This was not done in the original paper!

If the alphabet is larger, we can observe that the alphabet in the two substrings can be always normalised to be of size B (do you see why and how to do this efficiently?), so the number of different situations increases to B^{2B+2} , hence we should take $B = \Theta(\frac{\log n}{\log \log n})$.

Question

Can we significantly improve on the $O(n^2)$ time algorithm for computing LCS/edit distance?

Answer

Not really...

How to prove such a statement?

CNF-SAT

Boolean variables $x_1, \dots, x_N$ and clauses $C_1, \dots, C_M$, each being an OR over literals. Decide whether there is an assignment of $x_1, \dots, x_N$ that satisfies all the clauses.

k -SAT

Each clause consists of $\leq k$ literals.

Question

Can we significantly improve on the $O(n^2)$ time algorithm for computing LCS/edit distance?

Answer

Not really...

How to prove such a statement?

CNF-SAT

Boolean variables $x_1, \dots, x_N$ and clauses $C_1, \dots, C_M$, each being an OR over literals. Decide whether there is an assignment of $x_1, \dots, x_N$ that satisfies all the clauses.

k -SAT

Each clause consists of $\leq k$ literals.

Question

Can we significantly improve on the $O(n^2)$ time algorithm for computing LCS/edit distance?

Answer

Not really...

How to prove such a statement?

CNF-SAT

Boolean variables $x_1, \dots, x_N$ and clauses $C_1, \dots, C_M$, each being an OR over literals. Decide whether there is an assignment of $x_1, \dots, x_N$ that satisfies all the clauses.

k -SAT

Each clause consists of $\leq k$ literals.

Question

Can we significantly improve on the $O(n^2)$ time algorithm for computing LCS/edit distance?

Answer

Not really...

How to prove such a statement?

CNF-SAT

Boolean variables $x_1, \dots, x_N$ and clauses $C_1, \dots, C_M$, each being an OR over literals. Decide whether there is an assignment of $x_1, \dots, x_N$ that satisfies all the clauses.

k -SAT

Each clause consists of $\leq k$ literals.

Question

Can we significantly improve on the $O(n^2)$ time algorithm for computing LCS/edit distance?

Answer

Not really...

How to prove such a statement?

CNF-SAT

Boolean variables $x_1, \dots, x_N$ and clauses $C_1, \dots, C_M$, each being an OR over literals. Decide whether there is an assignment of $x_1, \dots, x_N$ that satisfies all the clauses.

k -SAT

Each clause consists of $\leq k$ literals.

Conjectures

$P \neq NP$

3-SAT cannot be solved in polynomial time.

implied by:

Exponential Time Hypothesis (ETH)

3-SAT cannot be solved in $O(2^{o(N)})$.

implied by:

Strong Exponential Time Hypothesis (SETH)

For every $\epsilon > 0$, there exists $k \geq 3$ such that k -SAT cannot be solved in $O(2^{(1-\epsilon)N})$.

known algorithms for k -SAT work in $O(2^{(1-c_k)n})$ time, $c_k = \Theta(1/k)$.

Conjectures

$P \neq NP$

3-SAT cannot be solved in polynomial time.

implied by:

Exponential Time Hypothesis (ETH)

3-SAT cannot be solved in $O(2^{o(N)})$.

implied by:

Strong Exponential Time Hypothesis (SETH)

For every $\epsilon > 0$, there exists $k \geq 3$ such that k -SAT cannot be solved in $O(2^{(1-\epsilon)N})$.

known algorithms for k -SAT work in $O(2^{(1-c_k)n})$ time, $c_k = \Theta(1/k)$.

Conjectures

$P \neq NP$

3-SAT cannot be solved in polynomial time.

implied by:

Exponential Time Hypothesis (ETH)

3-SAT cannot be solved in $O(2^{o(N)})$.

implied by:

Strong Exponential Time Hypothesis (SETH)

For every $\epsilon > 0$, there exists $k \geq 3$ such that k -SAT cannot be solved in $O(2^{(1-\epsilon)N})$.

known algorithms for k -SAT work in $O(2^{(1-c_k)n})$ time, $c_k = \Theta(1/k)$.

Sparsification Lemma

In k -SAT we can assume that there are N^k clauses. However, we can guarantee even more:

Impagliazzo, Paturi, and Zane 2001

For any k -CNF formula on N variables and M clauses and any $\epsilon \in (0, 1)$ there is an algorithm that outputs in $2^{\epsilon n} \text{poly}(M)$ time an equivalent collection of $2^{\epsilon n}$ k -CNF formulas, each on the same n variables but with only $O(c_{k,\epsilon} n)$ clauses.

...the proof is not trivial!

Sparsification Lemma

In k -SAT we can assume that there are N^k clauses. However, we can guarantee even more:

Impagliazzo, Paturi, and Zane 2001

For any k -CNF formula on N variables and M clauses and any $\epsilon \in (0, 1)$ there is an algorithm that outputs in $2^{\epsilon n} \text{poly}(M)$ time an equivalent collection of $2^{\epsilon n}$ k -CNF formulas, each on the same n variables but with only $O(c_{k,\epsilon} n)$ clauses.

...the proof is not trivial!

Sparsification Lemma

In k -SAT we can assume that there are N^k clauses. However, we can guarantee even more:

Impagliazzo, Paturi, and Zane 2001

For any k -CNF formula on N variables and M clauses and any $\epsilon \in (0, 1)$ there is an algorithm that outputs in $2^{\epsilon n} \text{poly}(M)$ time an equivalent collection of $2^{\epsilon n}$ k -CNF formulas, each on the same n variables but with only $O(c_{k,\epsilon} n)$ clauses.

...the proof is not trivial!

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

Williams 2005

SETH implies that, for every $\epsilon > 0$, there is no $O(n^{2-\epsilon})$ time algorithm for OV when $d = \omega(\log n)$.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

Williams 2005

SETH implies that, for every $\epsilon > 0$, there is no $O(n^{2-\epsilon})$ time algorithm for OV when $d = \omega(\log n)$.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

- 1 Take a k -SAT formula and split variables into $U = \{x_1, \dots, x_{N/2}\}$ and $V = \{x_{N/2+1}, \dots, x_N\}$.
- 2 For every assignment α to U define a binary vector A_α such that $A_\alpha[j] = 1$ iff α does not satisfy C_j .
- 3 For every assignment β to V define a binary vector B_β such that $B_\beta[j] = 1$ iff β does not satisfy C_j .
- 4 Now α, β satisfies the formula iff for every j either α or β satisfies C_j , or in other words A_α and B_β are orthogonal.
- 5 If we have an $O(n^{2-\epsilon})$ algorithm for OV then we have such an algorithm in dimension $d = c \log n$, for any constant c .
- 6 Set $c = c_{k, \epsilon/4}$ to obtain $O(2^{\epsilon/4N} (2^{N/2})^{2-\epsilon}) = O(2^{N(1-\epsilon/4)})$ algorithm.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

- 1 Take a k -SAT formula and split variables into $U = \{x_1, \dots, x_{N/2}\}$ and $V = \{x_{N/2+1}, \dots, x_N\}$.
- 2 For every assignment α to U define a binary vector A_α such that $A_\alpha[j] = 1$ iff α does not satisfy C_j .
- 3 For every assignment β to V define a binary vector B_β such that $B_\beta[j] = 1$ iff β does not satisfy C_j .
- 4 Now α, β satisfies the formula iff for every j either α or β satisfies C_j , or in other words A_α and B_β are orthogonal.
- 5 If we have an $O(n^{2-\epsilon})$ algorithm for OV then we have such an algorithm in dimension $d = c \log n$, for any constant c .
- 6 Set $c = c_{k, \epsilon/4}$ to obtain $O(2^{\epsilon/4 N} (2^{N/2})^{2-\epsilon}) = O(2^{N(1-\epsilon/4)})$ algorithm.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

- 1 Take a k -SAT formula and split variables into $U = \{x_1, \dots, x_{N/2}\}$ and $V = \{x_{N/2+1}, \dots, x_N\}$.
- 2 For every assignment α to U define a binary vector A_α such that $A_\alpha[j] = 1$ iff α does not satisfy C_j .
- 3 For every assignment β to V define a binary vector B_β such that $B_\beta[j] = 1$ iff β does not satisfy C_j .
- 4 Now α, β satisfies the formula iff for every j either α or β satisfies C_j , or in other words A_α and B_β are orthogonal.
- 5 If we have an $O(n^{2-\epsilon})$ algorithm for OV then we have such an algorithm in dimension $d = c \log n$, for any constant c .
- 6 Set $c = c_{k, \epsilon/4}$ to obtain $O(2^{\epsilon/4 N} (2^{N/2})^{2-\epsilon}) = O(2^{N(1-\epsilon/4)})$ algorithm.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

- 1 Take a k -SAT formula and split variables into $U = \{x_1, \dots, x_{N/2}\}$ and $V = \{x_{N/2+1}, \dots, x_N\}$.
- 2 For every assignment α to U define a binary vector A_α such that $A_\alpha[j] = 1$ iff α does not satisfy C_j .
- 3 For every assignment β to V define a binary vector B_β such that $B_\beta[j] = 1$ iff β does not satisfy C_j .
- 4 Now α, β satisfies the formula iff for every j either α or β satisfies C_j , or in other words A_α and B_β are orthogonal.
- 5 If we have an $O(n^{2-\epsilon})$ algorithm for OV then we have such an algorithm in dimension $d = c \log n$, for any constant c .
- 6 Set $c = c_{k,\epsilon/4}$ to obtain $O(2^{\epsilon/4N} (2^{N/2})^{2-\epsilon}) = O(2^{N(1-\epsilon/4)})$ algorithm.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

- 1 Take a k -SAT formula and split variables into $U = \{x_1, \dots, x_{N/2}\}$ and $V = \{x_{N/2+1}, \dots, x_N\}$.
- 2 For every assignment α to U define a binary vector A_α such that $A_\alpha[j] = 1$ iff α does not satisfy C_j .
- 3 For every assignment β to V define a binary vector B_β such that $B_\beta[j] = 1$ iff β does not satisfy C_j .
- 4 Now α, β satisfies the formula iff for every j either α or β satisfies C_j , or in other words A_α and B_β are orthogonal.
- 5 If we have an $O(n^{2-\epsilon})$ algorithm for OV then we have such an algorithm in dimension $d = c \log n$, for any constant c .
- 6 Set $c = c_{k, \epsilon/4}$ to obtain $O(2^{\epsilon/4 N} (2^{N/2})^{2-\epsilon}) = O(2^{N(1-\epsilon/4)})$ algorithm.

..and one more conjecture

Orthogonal Vectors (OV)

Given n binary vectors $A_1, \dots, A_n$ and $B_1, \dots, B_n$ in dimension $d = \text{polylog}(n)$, check if $A_i \cdot B_j = 0$ for some i, j .

- 1 Take a k -SAT formula and split variables into $U = \{x_1, \dots, x_{N/2}\}$ and $V = \{x_{N/2+1}, \dots, x_N\}$.
- 2 For every assignment α to U define a binary vector A_α such that $A_\alpha[j] = 1$ iff α does not satisfy C_j .
- 3 For every assignment β to V define a binary vector B_β such that $B_\beta[j] = 1$ iff β does not satisfy C_j .
- 4 Now α, β satisfies the formula iff for every j either α or β satisfies C_j , or in other words A_α and B_β are orthogonal.
- 5 If we have an $O(n^{2-\epsilon})$ algorithm for OV then we have such an algorithm in dimension $d = c \log n$, for any constant c .
- 6 Set $c = c_{k, \epsilon/4}$ to obtain $O(2^{\epsilon/4N} (2^{N/2})^{2-\epsilon}) = O(2^{N(1-\epsilon/4)})$ algorithm.

LCS*

Two strings over $\{0, 1, ?\}$, where $?$ is a don't care character. We want to find a longest common **substring**.

Note that this is not LCS considered last week. But, there is no obvious subquadratic time algorithm.

SETH implies that LCS* cannot be solved in $O(n^{2-\epsilon})$ time, for every $\epsilon > 0$.

Later we will see that without $?$ there is a **linear-time** solution.

LCS*

Two strings over $\{0, 1, ?\}$, where $?$ is a don't care character. We want to find a longest common **substring**.

Note that this is not LCS considered last week. But, there is no obvious subquadratic time algorithm.

SETH implies that LCS* cannot be solved in $O(n^{2-\epsilon})$ time, for every $\epsilon > 0$.

Later we will see that without $?$ there is a **linear-time** solution.

LCS*

Two strings over $\{0, 1, ?\}$, where $?$ is a don't care character. We want to find a longest common **substring**.

Note that this is not LCS considered last week. But, there is no obvious subquadratic time algorithm.

SETH implies that LCS* cannot be solved in $O(n^{2-\epsilon})$ time, for every $\epsilon > 0$.

Later we will see that without $?$ there is a **linear-time** solution.

LCS*

Two strings over $\{0, 1, ?\}$, where $?$ is a don't care character. We want to find a longest common **substring**.

Note that this is not LCS considered last week. But, there is no obvious subquadratic time algorithm.

SETH implies that LCS* cannot be solved in $O(n^{2-\epsilon})$ time, for every $\epsilon > 0$.

Later we will see that without $?$ there is a **linear-time** solution.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

$01x_k01$ where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

$?1y_k?1$ where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111 .

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

$01x_k01$ where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

$?1y_k?1$ where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111 .

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

$01x_k01$ where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

$?1y_k?1$ where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111 .

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

$01x_k01$ where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

$?1y_k?1$ where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111 .

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

01 x_k 01 where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

?1 y_k ?1 where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111.

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

$01x_k01$ where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

$?1y_k?1$ where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111 .

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

$01x_k01$ where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

$?1y_k?1$ where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111 .

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.

LCS*

We reduce OV. The strings are constructed as follows:

$$S = S_1 \$ S_2 \$ \dots \$ S_n$$
$$T = T_1 \# T_2 \# \dots \# T_n$$

S_i encodes A_i and T_j encodes B_j . The vectors are encoded coordinate-by-coordinate as follows:

01 x_k 01 where $x_k = 1$ when $S_i[k] = 0$ and $x_k = 0$ otherwise.

?1 y_k ?1 where $y_k = ?$ when $T_j[k] = 0$ and $y_k = 1$ otherwise.

LCS* is of length $5d$ iff there exists an orthogonal pair of vectors.

To reduce the alphabet, replace $\$$ with 000 and $\#$ with 111.

Subquadratic algorithm for LCS* implies subquadratic algorithm for OV.